CSC311: Introduction to Machine Learning

Allison Lau

Summer 2023

Contents

1 Introduction **1**
1.1 Input Vectors . . . 1
1.2 Nearest Neighbours . . . 2

2 Decision Trees **5**
2.1 Entropy . . . 5
2.2 Information Gain . . . 6
2.3 Decision Tree Construction . . . 7

3 Generalization and Bias-Variance Decomposition **9**
3.1 Generalization . . . 9
3.2 Bias-Variance Decomposition . . . 10
3.3 Bagging . . . 12

4 Linear Regression **14**
4.1 Introduction . . . 14
4.2 Vectorization . . . 14
4.3 Optimization . . . 15
4.4 Feature Mapping . . . 18
4.5 Regularization . . . 19

5 Binary Linear Classifiers **20**
5.1 Data Space and Weight Space . . . 21
5.2 The Perceptron Learning Rule . . . 21
5.3 The Limits of Linear Classifiers . . . 22
5.4 Choosing a Cost Function for the Binary Classifier . . . 23
5.4.1 0-1 loss . . . 23
5.4.2 Linear Regression . . . 23
5.4.3 Logistic Nonlinearity . . . 24
5.4.4 Logistic Regression . . . 25
5.4.5 Support Vector Machine (SVM) . . . 26
5.5 Multiclass Classifiction . . . 26
5.6 Convex Functions . . . 27
5.7 Tracking Accuracy for Binary Classification . . . 28

5.8 Gradient Checking with Finite Differences . . . 28
5.9 Linear Classifiers v.s kNN . . . 29

6 Neural Networks **30**
6.1 Multilayer Perceptrons . . . 30
6.2 Feature Learning . . . 32
6.3 Expressive Power . . . 33
6.4 Back Propoagation . . . 33

A Probability **35**

B Linear Algebra **36**

C Regularized Linear Regression **36**

1 Introduction

For many problems, such as recognizing people and objects, and understanding human speech, it is difficult to program the correct behaviour by hand. In other cases, systems need to adapt to a changing environment (e.g. spam detection), or that we want the system to perform better than human programmers, or even for privacy or fairness reasons. A machine learning algorithm automatically learns from data or experience.

Machine learning and statistics rely on similar mathematics but they differ in their goals. Statistics is more concerned with helping scientists and policymakers draw good conclusions, whereas machine learning is more concerned with building autonomous agents. Statistics puts more emphasis on interpretability and mathematical rigor, whereas machine learning puts on emphasis on predictive performance, scalability and autonomy.

Types of machine learning include:

Supervised learning: The system takes in inputs and labels/targets of the correct behaviour

Unsupervised learning: The system takes in inputs and look for "interesting" patterns in the data

Reinforcement learning: A learning system (agent) interacts with the world and learns to maximize a scalar reward signal

For much of this set of notes, we focus on supervised learning.

Examples of machine learning include computer vision, speech to text, natural language processing (machine translation, sentiment analysis, topic modeling, spam filtering), e-commerce and recommender systems, etc.

1.1 Input Vectors

Machine learning algorithms need to handle lots of types of data: images, text, audio waveforms, etc. Hence the common stategy is to represent the input as an ***input vector*** in $\mathbb{R}^d$. The mapping to another space that is easy to manipulate is known as ***representation***.

Mathematically, the training set consists of a collection of pairs of an input vector $\mathbf{x} \in \mathbb{R}^d$ and its corresponding ***target***, or ***label***, t:

Regression: t is a real number (e.g. stock price)

Classification: t is an element of a discrete set $\{1, ..., C\}$

We denote the traning set $\{(\mathbf{x}^{(1)}, t^{(1)}), ..., (\mathbf{x}^{(N)}, t^{(N)})\}$.

1.2 Nearest Neighbours

We start our discussion of machine learning algorithms with ***nearest neighbours***, which is a quasi-machine learning algorithm, or we say ***instance-based algorithm***, as there is no actual "learning" involved.

Suppose we are given a novel input vector $\mathbf{x}^*$ to classify. The idea is to find the nearest input vector to $\mathbf{x}$ in the training set and copy its label. We can formalize "nearest" in terms of Euclidean distance, which is given by[1]

$$||\mathbf{x}^{(a)} - \mathbf{x}^{(b)}||_2 = \sqrt{\sum_{j=1}^{d} (x_j^{(a)} - x_j^{(b)})^2}$$

Algorithm (NN):

1. Find $(\mathbf{x}^*, t^*)$ from the training set closest to $\mathbf{x}$:

$$\mathbf{x}^* = \underset{\mathbf{x}^{(i)} \in \text{training set}}{\text{argmin}} \quad \text{distance}(\mathbf{x}^{(i)}, \mathbf{x})$$

2. Output $y = t^*$

A ***Voronoi diagram*** is a type of geometric diagram that partitions a plane into regions based on the distance to a set of points called the seeds or generators. In a Voronoi diagram, each region is defined as the set of all points in the plane that are closer to a particular seed point than any other seed point. The boundaries between the regions are formed by lines that are equidistant between adjacent seed points. The ***decision boundary*** is the boundary between regions of input space assigned to different categories.

However, the method of nearest neighbours is sensitive to noise or mislabeled data ("class noise"). For example, if the nearest neighbour is misclassified, then the prediction we would like to obtain would be misclassified as well. We therefore smooth the noise by having k nearest neighbours vote.

[1] For more types of distances, see Appendix B

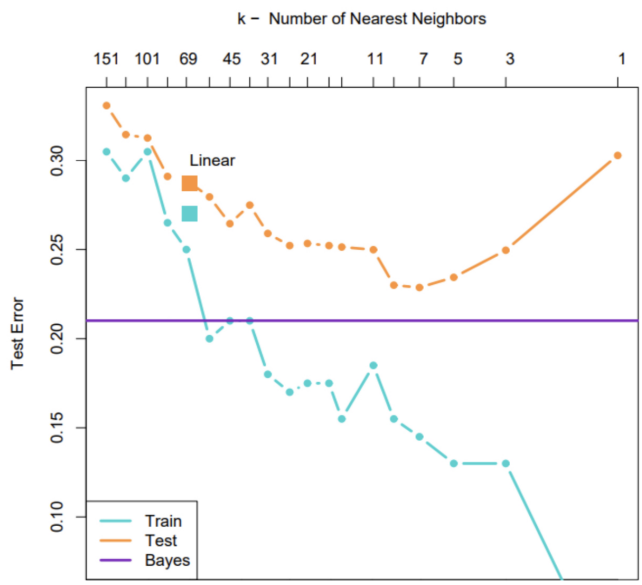

Figure 1: Test error in training set and test set with decreasing k

Algorithm (kNN):
1. Find k pairs $\{\mathbf{x}^{(i_1)}, t^{(i_1)}\}, \{\mathbf{x}^{(i_2)}, t^{(i_2)}\}, ..., \{\mathbf{x}^{(i_k)}, t^{(i_k)}\}$
2. Output the majority class

$$y^* = \underset{t^{(z)} \in \text{ class labels}}{\operatorname{argmax}} \sum_{i=1}^{k} \delta(t^{(z)} = t^{(i)})$$

There are tradeoffs whether we choose small k or large k. Small k is good at capturing fine-grained patterns but may ***overfit***, i.e. be sensitive to random idiosyncracies in the training data. Large k makes stable predictions by averaging lots of examples but may ***underfit***, i.e. fail to capture important regularities. The optimal choice of k depends on number of data points n. We get "nice theoretical properties" if $k \to \infty, \frac{k}{n} \to 0$, i.e. as the number of neighbors considered becomes much larger compared to the size of the training set, the model's bias increases and the variance decreases (the discussion of which we will leave to a later section). In other words, the model becomes less influenced by individual data points and more driven by the overall distribution and structure of the data. The rule of thumb is to choose $k < \sqrt{n}$. But if n is big, then we have a lot of k's to choose from. We can choose k using a ***validation set***.

We would like our algorithm to generalize to data it hasn't seen before. A ***test set*** can be used to measure the ***generalization error*** (error rate on new examples).

A ***hyperparameter*** is a parameter whose value is set before the learning process begins and remains constant during training. Unlike model parameters, which are learned from the training data, hyperparameters are typically chosen by the practitioner or determined through a process called hyperparameter tuning. k is a hyperparameter in this case.

In Figure 1, the training error of the training set is highest when k is highest and lowest when k is lowest, as it is fitting to the noise. In contrast, for the test set, there is a minimum test

error at a certain value of k. We therefore choose this k for our model. The set of data used for this purpose is often called the validation set, and we typically split our data into three portions — training set, validation set, and test set. The ideal model has low training error and test error.

Even if we chosen our k well, the method of k-Nearest Neighbours still has certain pitfalls:

i. **The Curse of Dimensionality.** High dimensional data can have negative impact on the performance and effectiveness of the algorithm.

   Increased data sparsity: The volume of a single ball of radius ϵ around each point is $\mathcal{O}(\epsilon^d)$. The total volume of $[0,1]^d$ is 1. Therefore $\mathcal{O}((\frac{1}{\epsilon})^d)$ points are needed to cover the volume, making it more difficult to identify neighboring points that are close in proximity. The reduced density makes it challenging to define a clear neighborhood around a given data point.

   A saving grace is that some datasets may have low ***intrinsic dimension***, i.e. lie on or near a low-dimensional manifold.

ii. **Computation Cost.** The number of computations at training time is 0, but the number of computations at test time per query is

   * Calculate D-dimensional Euclidean distance with N data points: $\mathcal{O}(ND)$
   * Sort the distances: $\mathcal{O}(N \log N)$

   which is very expensive by standards of a learning algorithm. Also, we need to store the entire dataset in memory.

iii. **Normalization.** In kNN, the distances between different data points are used to determine their similarity. When the ranges of different dimensions are not comparable or on different scales, there could be biased results, as the feature with a larger range may dominate the distance calculation.

   A fix to this is to ***normalize*** each dimension to zero mean and unit variance, i.e. to take

$$\widetilde{x_j} = \frac{x_j - \mu_j}{\sigma_j}$$

2 Decision Trees

A deicision tree is a tree with internal nodes that test a feature, branches determined by the feature value, and leaf nodes that represent outputs / final predictions. Thinking of the decision tree as partition, each path from the root to a leaf defines a region R_m of input space. We denote the training examples that fall into R_m to be $\{(x^{(m_1)}, t^{(m_1)}), ..., (x^{(m_1)}, t^{(m_1)})\}$.

For regression trees, the output is continuous, and the leaf value y_m is typically set to the mean value in $\{t^{(m_1)}, ..., t^{(m_k)}\}$. For classification trees, the output is discrete, and the leaf value y_m is typically set to the most common value in $\{t^{(m_1)}, ..., t^{(m_k)}\}$.

Decision trees are universal function approximators, meaning that they can approximate any arbitrary function with a given level of accuracy. Finding the smallest decision tree that correctly classifies a training set is NP-complete.[2] We therefore resort to a ***greedy heuristic***:

* Start with the whole training set and an empty decision tree
* Pick a feature and candidate split that would most reduce a loss (a scalar number to assess whether we are on track, the lower the loss the better the split.)
* Split on that feature and recurse on subpartitions

2.1 Entropy

To quantify uncertainty in prediction for a given leaf node, if all examples in the leaf node have the same class, then it is good because the uncertainty is low. If each class has the same amount of examples in a given leaf node, then it is bad because the uncertainty is high. We therefore need a small detour to information theory.

The ***entropy*** of a discrete random variable is the uncertainty inherent in its possible outcomes, with unit bit. The system whose outcomes are more certain has lower entropy (e.g. A loaded coin has a lower entropy than a fair coin). We can also think of entropy as the expected information content of a random draw

You cannot store the outcome of a random draw using fewer expected bits than the entropy without losing information.

The ***entropy*** of a discrete random variable Y is given by

$$H(Y) = -\sum_{y \in Y} p(y) \log_2 p(y)$$

[2] "Nondeterministic polynomial-time completeness." (NP-completeness) is a concept in computational complexity theory that classifies computational problems based on their difficulty and solvability. A computational problem is considered NP-complete if it belongs to the class of NP problems and has the property that any other problem in NP can be reduced to it in polynomial time. In other words, an NP-complete problem is one for which a solution can be verified in polynomial time, and any problem in NP can be transformed into it efficiently. The significance of NP-completeness lies in the fact that if a problem is proven to be NP-complete, it implies that solving that problem efficiently (in polynomial time) is unlikely. In other words, no efficient algorithm is known to solve all instances of an NP-complete problem.

For multivariables X and Y,

$$H(X,Y) = -\sum_{y \in Y} \sum_{x \in X} p(x,y) \log_2 p(x,y)$$

Hence we also have the conditional entropy

$$H(Y|X=x) = -\sum_{y \in Y} p(y|x) \log_2 p(y|x)$$

We can use $p(y|x) = \frac{p(x,y)}{p(x)}$ and $p(x) = \sum_y p(x,y)$ in further calculations.

Also the expected conditional entropy

$$\mathbb{E}[H(Y|X)] = \mathbb{E}_x[H(Y|X=x)] = \sum_{x \in X} p(x) H(Y|X=x) = -\sum_{x \in X} \sum_{y \in Y} p(x,y) \log_2 p(y|x)$$

Some useful properties of entropy include

* H is always non-negative
* Chain rule: $H(X,Y) = H(X|Y) + H(Y) = H(Y|X) + H(X)$
* If X and Y are independent, then X does not affect our uncertainty about Y: $H(Y|X) = H(Y)$
* Knowing Y makes our knowledge of Y certain: $H(Y|Y) = 0$
* Knowing X decreases only uncertainty about Y: $H(Y|X) \leq H(Y)$

High entropy indicates:

* Variable has a uniform like distribution over many outcomes
* Flat histogram
* Value sampled from it are more predictable

Low entropy indicates:

* Distribution is concentrated on only a few outcomes
* Histogram is concentrated in a few areas
* Values sampled from it are more predictable

2.2 Information Gain

Given two random variables Y and X, the ***information gain*** in Y due to X, or the ***mutual information*** of Y and X is given by

$$IG(Y|X) = H(Y) - H(Y|X)$$

In the context of building a decision tree (see the next section),

$$IG(Y) = H(S) - \sum_{y \in Y} p(y) H(S|y)$$

where Y is the feature, y are the possible values of Y, and S is the target variable (e.g. the random fruit in classifying fruits).

That is, we can interpret $H(Y)$ as the uncertainty in Y, and $H(Y|X)$ as the expected uncertainty that would remain in Y after seeing X. Note that if X is completely uninformative about Y, then $IG(Y|X) = 0$. If X is completely informative about Y, then $IG(Y|X) = H(Y)$. The information gain of a split refers to how much information (over the training set) about the class label Y is gained by knowing which side of a split we are on.

A simple example illustrates this:

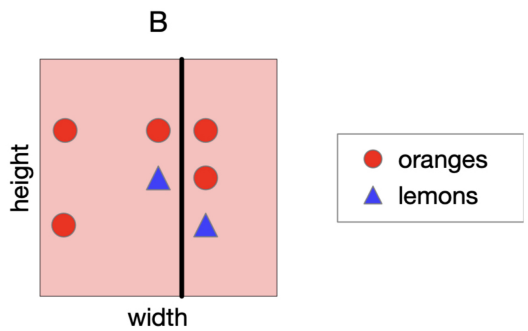

Figure 2: Splitting the training set based on width - first option

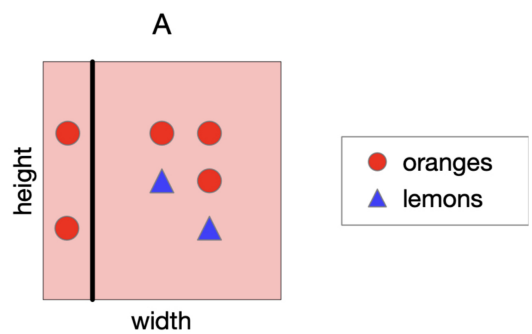

Figure 3: Splitting the training set based on width - second option

Let Y denote the random fruit ({oranges, lemons}), and let X denote the random side of the split ({left, right}).

Using the split on the left, $H(Y) = -\frac{2}{7}\log_2(\frac{2}{7}) - \frac{5}{7}\log_2(\frac{5}{7}) \approx 0.86$. $H(Y|\text{left}) \approx 0.81$, $H(Y|\text{right}) \approx 0.92$. $IG(\text{split}) = 0.86 - (\frac{4}{7} \cdot 0.81 + \frac{3}{7} \cdot 0.92) \approx 0.006$.

Using the split on the right, $H(Y) = -\frac{2}{7}\log_2(\frac{2}{7}) - \frac{5}{7}\log_2(\frac{5}{7}) \approx 0.86$. $H(Y|\text{left}) = 0$, $H(Y|\text{right}) \approx 0.97$. $IG(\text{split}) = 0.86 - (\frac{2}{7} \cdot 0 + \frac{5}{7} \cdot 0.97) \approx 0.17$.

As we can see the information gain of the second option is much higher the first. We therefore choose the split the second option as our split. Constructing a decision tree is similar in that at each level, the algorithm chooses the feature that gives the highest gain.

2.3 Decision Tree Construction

The decision tree construction uses a simple, greedy (i.e. makes locally optimal decisions at each step without considering the global optimal solution), recursive approach to build up tree node-by-node.

Algorithm (Decision Tree Construction):

1. Start with the entire dataset as the root node of the decision tree.
2. Calculate the information gain for all features (including the ones used higher up in the tree) in the current node. Select the attribute with the highest information gain as the splitting criterion for the current node.
3. Create a branch from the current node for each possible value of the selected attribute. Divide the instances in the current node into subsets.
4. At the node,
   * If all the instances in the current node belong to the same class, create a leaf node with that class label.
   * If there are no more attributes to split on, create a leaf node with the majority class label of the instances in the current node.
   * Else, recursively apply the algorithm to each subset.
5. Return the decision tree.

An uncontrolled decision tree will grow indefinitely, and easily overfit the data. Therefore, we use hyperparameters, such as limiting the size of the training set, or the number of levels the tree could have, to best generalize the patterns in the data. An ideal decision tree is:

Not too small:

* We need to handle important but possibly subtle distinctions in data

Not too big:

* Computational efficiency (avoid redundant, spurious attributes)
* Avoid over-fitting training examples
* Human interpretability

When constructing a decision tree, ***Occam's razor*** suggests favoring simpler, more concise trees over complex ones. This means that when evaluating different splits at each node, the algorithm should prioritize attributes that provide the most significant information gain while keeping the tree as simple as possible.

Just as any machine learning algorithm, the decision tree has certain miscellanies. We may have exponentially less data at lower levels. Also, greedy algorithms don't necessarily yield the global optimum. When handling continuous attributes, we split based on a threshold, chosen to maximize information gain. Decision trees can also be used for regression on real-valued outputs. Choose splits to minimize squared error, rather than maximize information gain.

Although compared with k-NN, decision trees are simple to deal with discrete features, missing values, and poorly scaled data, fast at test time, and more interpretable, it uses more hyperparameters and and incorporate interesting distance measures (e.g. shape context).

3 Generalization and Bias-Variance Decomposition

Let us now turn to study generalization and bias-variance decomposition, one of the most important concepts in machine learning.

3.1 Generalization

When we train a machine learning model, we want it to *generalize* to data it hasn't seen before. Recall partitioning the data into three subsets:

Training set, a set of training examples the model is training on.

Validation set, a set used to tune hyperparameters such as the number of hidden units, or the learning rate.

Test set, a set used to measure the generalization performance, used at the very end.

Note the difference between a validation set and a test set: The test set is only used to report the final performance. We cannot tune hyperparamters on the test set, because that would be "cheating". Neither can we tune hyperparameters on the training set – that could lead us to choose a very expressive model that memorize the training data, which generalizes poorly.

There are two strategies for tuning hyperparamters: ***grid search*** and ***random search***, which differ in how the hyperparamters are chosen.

Grid search: For each hyperparameter, choose a set of candidate values. Separately train models using all possible combinations of these values.

Random search: Separately train models using combinations of random hyperparameter.

Then choose the configuration which gives the smallest validation error.. The advantage of random search over grid search is as follows:

Exploration of Hyperparameter Space: Random search allows for a more comprehensive exploration of the hyperparameter space compared to grid search. In grid search, the hyperparameter values are predefined and limited to a fixed set of values, which may miss important regions of the hyperparameter space. Random search randomly samples values for each hyperparameter from a specified range or distribution, enabling a wider exploration of the space.

Efficiency in High-Dimensional Spaces: As the number of hyperparameters increases, the grid search becomes computationally expensive since it requires evaluating all possible combinations of hyperparameter values. Random search only requires a fixed number of random samples, making it more efficient in high-dimensional spaces.

Handling Irrelevant Hyperparameters: In many cases, some hyperparameters have a minimal impact on the performance of the model. Grid search evaluates all possible combinations, including those with irrelevant hyperparameters, which wastes computational resources. Random search, by its random nature, may skip or assign lower priority to such irrelevant hyperparameters, focusing more on the ones that have a significant impact.

Let us now think about how the training and generalization error carry as a function of the number of training examples and the number of parameters as shown in Figure 4.

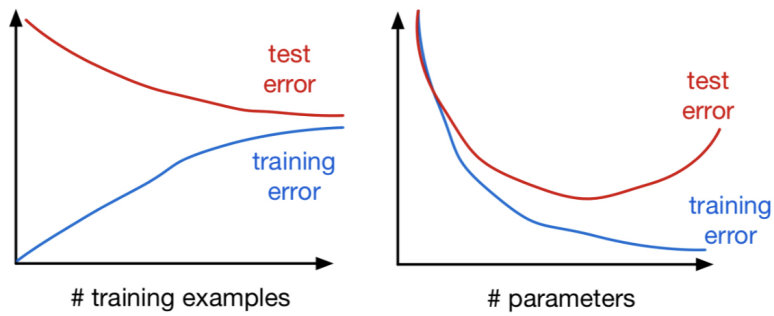

Figure 4: (left) Qualitative relationship between number of traning examples and training/test error. (right) Qualitative relationship between the number of parameters (or model capacity) and training/test error.

Consider the number of training examples. The larger the training set, the more likely there will be a closely related training example, and the fewer the ***accidental regularities***, so the model will be forced to pick up the true regularities, so generalization error decreases. On the other hand, larger training sets are harder to memorize than small training sets, so training error increases.

Now consider the number of model capacity.[3] The more the parameters, the easier it is to fit both the accidental and the true regularities of the training data, so training error decreases. The effect on the generalization error is more subtle. If the model has too little capacity, it generalizes badly because it fails to pick up regularities, i.e. It underfits. if it has too much capacity, it will memorize the training set and fail to generalize, so the generalization error decreases then increases.

3.2 Bias-Variance Decomposition

Now we would like to mathematically model the generalization error. We introduce the ***data generating distribution***, a hypothetical distribution $p_{\mathcal{D}}(\mathbf{x}, t)$ that all the training and test data are assumed to have come from. We don't assume anything about the form of the distribution. The only nontrivial assumption is that the training and test data are drawn from the same distribution. A training set $\mathcal{D}$ consists of pairs $(\mathbf{x}_i, t_i)$ sampled ***independent and identically distributed (i.i.d.)*** from $p_{\mathcal{D}}(\mathbf{x}, t)$.

In quantifying the generlization error, what we would really like to decompose is the expectation of the squared error loss,

$$L(y, t) = \frac{1}{2}(y - t)^2$$

where y is the prediction at a query point $\mathbf{x}$ and t is a sampled from a conditional distribution $p(t|x)$, and $y - t$ is known as the ***residual***.

[3]Capacity isn't a formal term, but corresponds roughly to the number of trainable parameters.

Basic setup (Bias-Variance Decomposition):

1. Fix a query point $\mathbf{x}$
2. Sample multiple training sets $\mathcal{D}$ i.i.d. from the data generating distribution $p_{\mathcal{D}}(\mathbf{x}, t)$
3. For each training set $\mathcal{D}$,
   * Run the learning algorithm on $\mathcal{D}$ to get a prediction y at $\mathbf{x}$
   * Sample a true target t from the conditional probability $p(t|x)$.
   * Compute the loss $L(x, t)$

This gives us a distribution over the loss at $\mathbf{x}$, with expectation $\mathbb{E}[L(y, t)|x]$. We would like to minimize this expectation.

Let us now consider only t to be a random variable (as it is sampled from a conditional probability), and y to be a constant (consider a fixed training set). We claim that $y = \mathbb{E}[t|\mathbf{x}]$ is the best possible prediction. Here is a proof:

$$\begin{aligned}
\mathbb{E}[(y-t)^2|\mathbf{x}] &= \mathbb{E}[y^2 - 2yt + t^2|\mathbf{x}] \\
&= y^2 - 2y\mathbb{E}[t|\mathbf{x}] + \mathbb{E}[t^2|\mathbf{x}] \\
&= y^2 - 2y\mathbb{E}[t|\mathbf{x}] + (\mathbb{E}[t|\mathbf{x}])^2 + \mathrm{Var}[t|\mathbf{x}] \\
&= (y - \mathbb{E}[t|\mathbf{x}])^2 + \underbrace{\mathrm{Var}[t|x]}_{\text{Bayes error}}
\end{aligned}$$

This is minimized when $y = \mathbb{E}[t|\mathbf{x}]$. Notice that the ***Bayes error*** does not depend y. If the $\mathbb{E}[(y-t)^2|\mathbf{x}]$ is equal to the Bayes error, then the algorithm is ***Bayes optimal***. It is the best we can ever hope to do with any learning algorithm. The process of choosing y based on $p(t|\mathbf{x})$ is an example of decision theory.

Let us now treat y as a random variable, where the randomness comes from the choice of training set. Then we have

$$\begin{aligned}
\mathbb{E}[(y-t)^2|\mathbf{x}] &= \mathbb{E}[y^2 - 2yt + t^2|\mathbf{x}] \\
&= \mathbb{E}[y^2|\mathbf{x}] - 2\mathbb{E}[yt|\mathbf{x}] + \mathbb{E}[t^2|\mathbf{x}] \\
&= (\mathbb{E}[y|\mathbf{x}])^2 + \mathrm{Var}[y|x] - 2\mathbb{E}[y|\mathbf{x}]\mathbb{E}[t|\mathbf{x}] + (\mathbb{E}[t|\mathbf{x}])^2 + \mathrm{Var}[t|x] \quad \text{Since } y \text{ and } t \text{ are independent} \\
&= \underbrace{(\mathbb{E}[t|\mathbf{x}] - \mathbb{E}[y|\mathbf{x}])^2}_{\text{bias}} + \underbrace{\mathrm{Var}[y|\mathbf{x}]}_{\text{variance}} + \underbrace{\mathrm{Var}[t|\mathbf{x}]}_{\text{Bayes error}}
\end{aligned}$$

We have decomposed the expected loss into three terms, known as the ***bias-variance decomposition***:

* ***Bias***: How far off the model's average prediction is (corresponds to underfitting – The higher the bias, the worse the perfomance of the model)
* ***Variance***: Variability in the model's prediction across the training sets used (corresponds to overfitting – The higher the variance, the more the model overfits with the individual training sets)
* ***Bayes error***: The inherent unpredictability of the targets

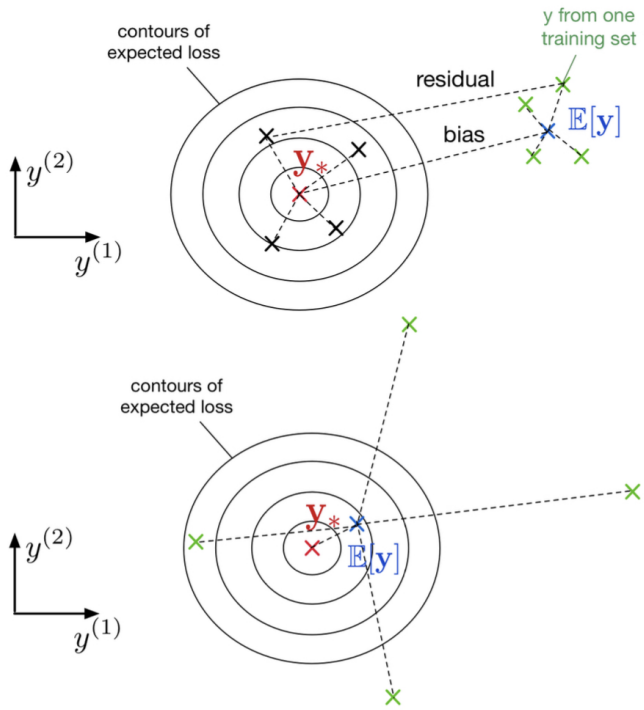

Figure 5: (top) If the model is underfitting, the bias is large but the variance is small. (bottom) If the model is overfitting, the bias is small but the variance is large

3.3 Bagging

In short, bagging reduces over-fitting by averaging predictions.

Consider the errors of using one training set and multiple training sets. Suppose we sample m independent training sets from p_{sample}, and compute the prediction y_i using each training set, obtaining the average prediction $y = \frac{1}{m}\sum_{i=1}^{m} y_i$. Consider how this affects the bias and variance.

The bias is unchanged: $\mathbb{E}[y] = \mathbb{E}\left[\frac{1}{m}\sum_{i=1}^{m} y_i\right] \approx \mathbb{E}[y_i]$, since the averaged prediction has the same expectation (and similarly for $\mathbb{E}[t]$). The variance is reduced: $\text{Var}[y] = \text{Var}\left[\frac{1}{m}\sum_{i=1}^{m} y_i\right] = \frac{1}{m^2}\sum_{i=1}^{m}\text{Var}[y_i] \approx \frac{1}{m}\text{Var}[y_i]$, since we are averaging over independent predictions.

Given training set $\mathcal{D}$, use the empirical distribution $p_{\mathcal{D}}$ as a proxy for p_{sample}. Note that as $|\mathcal{D}| \to \infty$, we have $p_{\mathcal{D}} \to p_{\text{sample}}$.

For binary classification (which we will discuss in a later section), we average the predictions and apply a threshold,

$$y_{\text{bagged}} = \mathbb{I}\left(\frac{1}{m}\sum_{i=1}^{m} y_i > 0.5\right)$$

just as majority vote.

Bagging / Bootstrap aggregation:

1. Take a training set $\mathcal{D}$ with n examples.
2. Generate m bootstrap samples, each containing n examples sampled from $\mathcal{D}$ with replacement.
3. Obtain the predictions of the models $\{y_1, ..., y_m\}$. Average the predictions of models training on the m dataset $= (1/m) \sum_{i=1}^{m} y_i$.

Random forests, one of the popular ensemble[4] methods, are an extension of bagging where the base models are decision trees. Random forests combine bagging with random feature selection to further enhance the diversity and robustness of the ensemble models.

A bagged classifier can be stronger than the average model. It also reduces correlation between datasets by introducing additional variability. But note that if m datasets are not independent, we don't get the $\frac{1}{m}$ variance reduction. Bagging has certain limitations. It does not reduce bias in case of squared error. Also, weighting members equally my not be the best. Bagging can be sensitive to the specific training data samples. Since each base model is trained on a different bootstrap sample, the ensemble's performance can vary depending on the random sampling. In some cases, certain samples or outliers can have a disproportionate influence on the ensemble's predictions. Further, there could be high correlation between classifiers, leading to less variance reduction.

[4]Ensemble is the set of trained models whose predictions we are combining

4 Linear Regression

The third learning algorithm we will look at is ***linear regression***. Linear regression exemplifies a modular approach that will be used throughout this set of notes:

* Choose a model decribing the relationship between variables of interest
* Define a ***loss function*** quantifying how bad the fit to the data is
* Choose a ***regularizer*** saying how much we prefer different candidate models
* Fit a model that minimizes the loss function and satisfies the constraint/penalty imposed by the regularizer, possibly using an ***optimization algorithm***.

4.1 Introduction

In linear regression, we use a linear function of the features $\mathbf{x} = (x_1, ..., x_D) \in \mathbb{R}^D$ to make predictions y of the target value $t \in \mathbb{R}$:

$$y = f(\mathbf{x}) = \sum_{j=1}^{D} w_j x_j + b$$

$$y = \mathbf{w}^\intercal \mathbf{x} + b$$

where y is the prediction, $\mathbf{w}$ is the weights, b is the bias.

The ***loss function*** $\mathcal{L}(y, t)$ quantifies the discrepancy between the predicted values and the true values. For linear regression, the loss function we would choose is the squared error loss function

$$\mathcal{L}(y, t) = \frac{1}{2}(y - t)^2$$

which we would like to minimize, and the $\frac{1}{2}$ factor is for calculation convenience.

Suppose we have N training examples. The ***cost function*** is the loss function averaged over all training examples and is defined to be

$$\mathcal{J}(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^{N} \left(y^{(i)} - t^{(i)}\right)^2 = \frac{1}{2N} \sum_{i=1}^{N} \left(\mathbf{w}^\intercal \mathbf{x}^{(i)} + b - t^{(i)}\right)^2$$

4.2 Vectorization

For our algorithm to be more efficient, we vectorize them by expressing them in terms of vectors and matrices. For the linear model $y = \mathbf{w}^\intercal \mathbf{x} + b$, it can be expressed in python simply by `y = np.dot(w, x) + b`.

We can organize all the training examples into a ***design matrix*** $\mathbf{X}$ with rows representing training examples, and columns representing features. We also organize the targets into the ***target vector*** $\mathbf{t}$

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}^{(1)\intercal} \\ \vdots \\ \mathbf{x}^{(N)\intercal} \end{pmatrix}$$

Computing the predictions for the whole dataset,

$$\mathbf{X}\mathbf{w} + b\mathbf{1} = \begin{pmatrix} \mathbf{w}^\intercal \mathbf{x}^{(1)} + b \\ \vdots \\ \mathbf{w}^\intercal \mathbf{x}^{(N)} + b \end{pmatrix} = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{pmatrix} = \mathbf{y}$$

Computing the squared error cose across the whole dataset,

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b\mathbf{1}$$

$$\mathcal{J} = \frac{1}{2N}||\mathbf{y} - \mathbf{t}||^2$$

We could also add a column of 1's to the design matrix and conveniently write

$$\mathbf{X} = \begin{bmatrix} 1 & [\mathbf{x}^{(1)}]^\intercal \\ \vdots & \vdots \\ 1 & [\mathbf{x}^{(N)}]^\intercal \end{bmatrix} \in \mathbb{R}^{N \times (D+1)} \quad \text{and} \quad \mathbf{w} = \begin{bmatrix} b \\ w_1 \\ \vdots \\ w_D \end{bmatrix} \in \mathbb{R}^{D+1}$$

Then our predictions reduce to $\mathbf{y} = \mathbf{X}\mathbf{w}$.

Vectorized code can be much faster than explicit `for`-loops, because of the following reasons,

* High-level languages like Python can introduce a lot of interpreter overhead, and if we explicitly write a `for`-loop, it could be 10 - 100 times slower than the C equivalent. If instead we write the algorithm in terms of a much smaller number of linear algebra operations, then it can perform the same computations much faster with minimal interpreter overhead.
* Linear algebra libraries have been extremely well optimized for various computer architectures. Hence, they use much more efficient memory-access patterns than a naïve `for`-loop.
* Matrix multiplication is inherently highly parallelizable and involve little control flow. Hence, it is ideal for ***graphics processing unit (GPU)*** architectures. Running a vectorized code with framneworks like TensorFlow or PyTorch, it may run 50 times faster than the CPU version.

4.3 Optimization

Our goal is to minimize the cost function $\mathcal{J}(\mathbf{x})$. Recall from calculus that the minimum of a smooth function, if it exists, occurs at a ***critical point***, i.e. where the derivative is zero. In particular, our squared error loss function has a global optimum because it is convex (which we will discuss in a later section).

$$\nabla_{\mathbf{w}} \mathcal{J} = \frac{\partial \mathcal{J}}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial \mathcal{J}}{\partial w_1} \\ \vdots \\ \frac{\partial \mathcal{J}}{\partial w_D} \end{pmatrix}$$

Solutions may be direct or iterative.

* ***Direct solution***: Set the gradient to zero and solve in closed form – directly find provably optimal parameters.

* ***Iterative solution***: Repeatedly apply an update rule that gradually takes us closed to the solution.

Let us look at the direct solution first.

We seek $\mathbf{w}$ that minimizes $\mathcal{J}(\mathbf{w}) = \frac{1}{2}||\mathbf{X}\mathbf{w} - \mathbf{t}||^2$. (We will ignore the N for convenience and clarity.) From $||\mathbf{x}||_2 = \mathbf{x}^\intercal\mathbf{x}$ and properties of the transpose,[5]

$$\begin{aligned}
||\mathbf{X}\mathbf{w} - \mathbf{t}||^2 &= (\mathbf{X}\mathbf{w} - \mathbf{t})^\intercal(\mathbf{X}\mathbf{w} - \mathbf{t}) \\
&= ((\mathbf{X}\mathbf{w})^\intercal - \mathbf{t}^\intercal)(\mathbf{X}\mathbf{w} - \mathbf{t}) \\
&= (\mathbf{X}\mathbf{w})^\intercal\mathbf{X}\mathbf{w} - (\mathbf{X}\mathbf{w})^\intercal\mathbf{t} - \mathbf{t}^\intercal\mathbf{X}\mathbf{w} + \mathbf{t}^\intercal\mathbf{t} \\
&= \mathbf{w}^\intercal\mathbf{X}^\intercal\mathbf{X}\mathbf{w} - (\mathbf{X}\mathbf{w})^\intercal\mathbf{t} - \mathbf{t}^\intercal\mathbf{X}\mathbf{w} + \mathbf{t}^\intercal\mathbf{t} \\
&= \mathbf{w}^\intercal\mathbf{X}^\intercal\mathbf{X}\mathbf{w} - 2\mathbf{t}^\intercal\mathbf{X}\mathbf{w} + \mathbf{t}^\intercal\mathbf{t} \quad \text{since } \mathbf{X}\mathbf{w} \text{ and } \mathbf{t} \text{ are vectors} \\
&= \mathbf{w}^\intercal\mathbf{X}^\intercal\mathbf{X}\mathbf{w} - 2(\mathbf{X}^\intercal\mathbf{t})^\intercal\mathbf{w} + \mathbf{t}^\intercal\mathbf{t}
\end{aligned}$$

So then we have[6]

$$\begin{aligned}
\nabla(\frac{1}{2}||\mathbf{X}\mathbf{w} - \mathbf{t}||^2) &= \frac{1}{2}(-2\mathbf{X}^\intercal\mathbf{t} + (\mathbf{X}^\intercal\mathbf{X} + (\mathbf{X}^\intercal\mathbf{X})^\intercal)\mathbf{w}) \\
&= -\mathbf{X}^\intercal\mathbf{t} + \mathbf{X}^\intercal\mathbf{X}\mathbf{w} \quad \text{since } (\mathbf{X}^\intercal\mathbf{X})^\intercal = \mathbf{X}^\intercal\mathbf{X} \\
&= \mathbf{X}^\intercal\mathbf{X}\mathbf{w} - \mathbf{X}^\intercal\mathbf{t}
\end{aligned}$$

Setting this to 0,

$$\begin{aligned}
\mathbf{X}^\intercal\mathbf{X}\mathbf{w} &= \mathbf{X}^\intercal\mathbf{t} \\
\mathbf{w} &= (\mathbf{X}^\intercal\mathbf{X})^{-1}\mathbf{X}^\intercal\mathbf{t} \quad \text{since } \mathbf{X}^\intercal\mathbf{X} \text{ is invertible}
\end{aligned}$$

However, many optimization problems do not have a direct solution. A more braodly applicable way to minimize the cost function is ***gradient descent***. Gradient descent is an ***iterative algorithm***, which means we apply an update repeatedly until some criterion is met. We initialize the weights to something reasonable, and repeatedly adjust them in the direction of steepest descent.

Notice that :

If $\partial\mathcal{J}/\partial w_i > 0$, then decreasing $\mathcal{J}$ requires decreasing w_i.

If $\partial\mathcal{J}/\partial w_i < 0$, then decreasing $\mathcal{J}$ requires increasing w_i

Consider a learning rate $\alpha > 0$. The larger α is, the faster $\mathbf{w}$ changes. If cost is the total loss rather than average loss, a smaller learning rate will be needed ($\alpha' = \alpha/N$). The following update always decreases the cost function for small enough α

$$w_i \leftarrow w_i - \alpha\frac{\partial\mathcal{J}}{\partial w_i}$$

Update rule in vector form:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha\frac{\partial\mathcal{J}}{\partial\mathbf{w}}$$

[5]For two matrices A and B, $(A+B)^\intercal = A^\intercal + B^\intercal$, $(AB)^\intercal = B^\intercal A^\intercal$

[6]See https://www.math.usm.edu/lambers/mat610/lecture8.pdf

Update rule for linear regression (see derivation in Appendix C):

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$$

Gradient descent updates $\mathbf{w}$ in the direction of fastest decrease. Once it converges, there is a critical point at $\mathbf{w}$.

We prefer gradient descent over direct solution, because of two main reasons:

* We can only solve the system of equations explicitly for a handful of models. By contrast, we can apply gradient descent to any model for which we can compute the gradient. This is usually easy to do efficiently and *automatically*. Software packages such as Theano and TensorFlow can save us from having to compute partial derivatives by hand.

* For example, the linear regression direct solution $(\mathbf{X}^\intercal \mathbf{X})^{-1} \mathbf{X}^\intercal \mathbf{t}$ requires matrix inversion, which takes $\mathcal{O}(D^3)$. Each gradient descent update costs $\mathcal{O}(ND)$ or less with stochastic gradient descent (which we will see in a later section). The difference is significant if $D \gg 1$. Therefore, gradient descent can sometimes find a reasonable solution much faster than solving the linear system.

Let us now discuss about the learning rate α. There are a number of problems about α. If α is too small, there would be slow convergence. If α is too large, there would be oscillations and we would miss the optimum. If α is much too large, there would be instability. Good values are typically between 0.001 and 0.1. We could do a grid search, and diagonose optimization problems using a training curve.

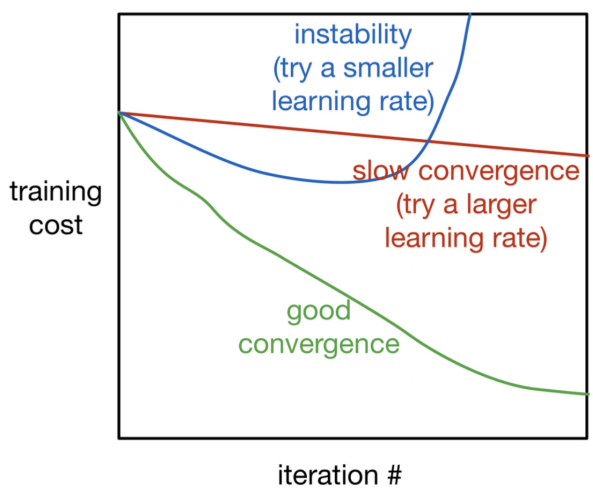

Figure 6: Training cost v.s iteration number (different lines represent choices of α)

There are three common types of gradient descent:

* ***Batch Gradient Descent***: The model parameters are updated based on the average gradient of the loss function computed over the entire training set. The cost function and gradient are respectively

$$\mathcal{J}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(y(\mathbf{x}^{(i)}, \boldsymbol{\theta}), t^{(i)})$$

$$\frac{\partial \mathcal{J}}{\partial \boldsymbol{\theta}} = \frac{1}{N} \sum_{i=1}^{N} \frac{\partial \mathcal{L}^{(i)}}{\partial \boldsymbol{\theta}}$$

where $\boldsymbol{\theta}$ denotes the parameters (e.g. in linear regression, $\boldsymbol{\theta} = (\mathbf{w}, b)$).

Batch gradient descent can be computationally expensive, especially for large datasets, as it requires processing the entire dataset in each iteration.

* ***Stochastic Gradient Descent***: The model parameters are updated based on the gradient of the loss function computed on a single training example chosen uniformly at random in each iteration.

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \frac{\partial \mathcal{L}^{(i)}}{\partial \boldsymbol{\theta}}$$

The benifits are that the cost of each update is independent of N, and we make significant progress before seeing all the data. Stochastic gradient is an unbiased estimate of the batch gradient given sampling each example uniformly at random.

$$\mathbb{E}\left[\frac{\partial \mathcal{L}^{(i)}}{\partial \theta}\right] = \frac{1}{N} \sum_{i=1}^{N} \frac{\partial \mathcal{L}^{(i)}}{\partial \theta} = \frac{\partial \mathcal{J}}{\partial \theta}$$

However, there is high variance in the estimate and we can't exploit efficient vectorized operations.

* ***Mini-Batch Gradient Descent***: Compute and update the model parameters using a randomly selected medium-sized subset of training examples $\mathcal{M}$, called ***mini-batch*** in each iteration. In theory, we sample examples independently and uniformly with replacement. In practice, we permute the training set and then go through it sequentially. A complete pass through the entire dataset is called an ***epoch***.

  $|\mathcal{M}|$ is a hyper-parameter. A reasonable value might be $|\mathcal{M}| = 100$. There are trade-offs for large and small mini-batch sizes: Large mini-batch size requires more computation time, estimates more accurately, and can exploit vectorization. Small mini-batch size updates faster, but estimates noisier and cannot exploit vectorization.

The learning rate influences the noise in the parameters from the stochastic updates. The typical strategy is to start with a large learning rate to get close to the optimum, then gradually decrease the learning rate to reduce the fluctuations.

4.4 Feature Mapping

Consider mapping the input features to another space using $\psi(\mathbf{x}) : \mathbb{R}^D \to \mathbb{R}^d$, then treating the mapped feature (in $\mathbb{R}^d$) as the input of a linear regression procecdure. Then we can use linear regression to model a non-linear relationship.

Consider that fitting the data using a degree-M polynomial function of the form

$$y = w_0 + w_1 x + w_2 x^2 + ... + w_M x^M = \sum_{i=0}^{M} w_i x^i$$

Choose the feature mapping is $\psi(x) = [1, x, x^2, ..., x^M]^\intercal$. $y = \psi(x)^\intercal \mathbf{w}$ is linear in $w_0, w_1, ..., w_M$. Hence we can use linear regression to find $\mathbf{w}$.

4.5 Regularization

The degree M of the polynomial controls the model's complexity. It is a hyperparameter for polynomial expansion and can be tuned using a validation set. Restricting the number of parameters / basis functions (M) is a crude approach to controlling the model complexity. Another approach is to keep the model large, but ***regularize*** it. A ***regularizer*** is a function that quantifies how much we prefer one hypothesis versus another. It encourages the model to have smaller coefficient values, leading to a simpler and more generalized model. This helps to reduce the risk of overfitting, where the model becomes too specific to the training data and performs poorly on new, unseen data. By adding this term to the loss function, the optimization process seeks to minimize both the data fitting error and the squared values of the coefficients.

Consider choosing the ℓ_2 penalty as our regularizer to encourage the weights to be small

$$\mathcal{R}(\mathbf{w}) = \frac{1}{2}||\mathbf{w}||_2^2 = \frac{1}{2}\sum_j w_j^2$$

The regularized cost function makes a tradeoff between the fit to the data and the norm of the weights.

$$\mathcal{J}_{\text{reg}}(\mathbf{w}) = \mathcal{J}(\mathbf{w}) + \lambda\mathcal{R}(\mathbf{w}) = \mathcal{J}(\mathbf{w}) + \frac{\lambda}{2}\sum_j w_j^2$$

Large λ penalizes weight values more. λ is a hyperparameter we can tune with a validation set.

Recall that for the least squares problem, we have $\mathcal{J}(\mathbf{w}) = \frac{1}{2N}||\mathbf{X}\mathbf{w} - \mathbf{t}||^2$. When $\lambda > 0$, regularized cost gives

$$\begin{aligned}
\nabla\mathcal{J}_{\text{reg}}(\mathbf{w}) &= \nabla(\frac{1}{2N}||\mathbf{X}\mathbf{w} - \mathbf{t}||^2 + \frac{\lambda}{2}||\mathbf{w}||_2^2) = 0 \\
\mathbf{w}_\lambda^{\text{Ridge}} &= (\mathbf{X}^\intercal\mathbf{X} + \lambda N\mathbf{I})^{-1}\mathbf{X}^\intercal\mathbf{t}
\end{aligned}$$

See Section 4.3 on Optimization for details of a similar derivation.

The gradient descent update to minimize the ℓ_2 regularized cost $\mathcal{J} + \lambda\mathcal{R}$ results in ***weight decay***:

$$\begin{aligned}
\mathbf{w} \leftarrow \mathbf{w} &- \alpha\frac{\partial}{\partial\mathbf{w}}(\mathcal{J} + \lambda\mathcal{R}) \\
&= \mathbf{w} - \alpha\left(\frac{\partial\mathcal{J}}{\partial\mathbf{w}} + \lambda\frac{\partial\mathcal{R}}{\partial\mathbf{w}}\right) \\
&= \mathbf{w} - \alpha\left(\frac{\partial\mathcal{J}}{\partial\mathbf{w}} + \lambda\mathbf{w}\right) \\
&= (1 - \alpha\lambda)\mathbf{w} - \alpha\frac{\partial\mathcal{J}}{\partial\mathbf{w}}
\end{aligned}$$

For a more specific example, see Appendix C.

5 Binary Linear Classifiers

We will be looking at classifiers which are both ***binary*** (they distinguish between two categories) and ***linear*** (the classification is done using a linear function of the inputs). The way binary linear classifiers work is simple: they compute a linear function of the inputs, and determine whether or not the value is larger than some ***threshold*** r. Recall that a linear function of the input can be written as

$$w_1x_1 + ... + w_Dx_D + b = \mathbf{w}^\intercal\mathbf{x} + b$$

where $\mathbf{w}$ is the weight vector and b is a scalar-valued bias. Therefore, with the threshold r, the prediction y can be computed as follows:

$$z = \mathbf{w}^\intercal\mathbf{x} + b$$
$$y = \begin{cases} 1 & \text{if } z \geq r \\ 0 & \text{if } z < r \end{cases}$$

Observe that

$$\mathbf{w}^\intercal\mathbf{x} + b \geq r \iff \mathbf{w}^\intercal\mathbf{x} + b - r \geq 0$$

We can obtain an equivalent model by replacing the bias $b - r$ and setting r to 0. Therefore we can rewrite the model

$$z = \mathbf{w}^\intercal\mathbf{x} + b$$
$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

We can also eliminate the bias by adding another input dimension x_0 called a ***dummy feature***, which always takes the value 1. The

$$w_0x_0 + w_1x_1 + ... + w_Dx_D = w_0 + w_1x_1 + ... + w_Dx_D \implies z = \mathbf{x}^\intercal\mathbf{w}$$

However, the standard practice when implementing the algorithm is to include the bias parameter explicitly.

Example 5.1 Let us consider representing *AND* using linear classifiers. The training cases are as follows:

x_1	x_2	t
0	0	0
0	1	0
1	0	0
1	1	1

We can write the inequalities corresponding to each training case.

$$\begin{aligned} b &< 0 \\ w_2 + b &< 0 \\ w_1 + b &< 0 \\ w_1 + w_2 + b &> 0 \end{aligned}$$

It is clear that $b < 0$ and $w_1, w_2 > 0$. Since the problem is symmetric with respect to w_1 and w_2, we might as well decide that $w_1 = w_2$. By trial and error, when we try $b = -1.5, w_1 = w_2 = 1$, all four inequalities are satisfied, so we have our solution.

5.1 Data Space and Weight Space

For the ***data space***, each point in the space corresponds to a possible input vector. (We are going to abuse mathematical terminology by using "point" and "vector" interchangeably.) Once we have chosen $\mathbf{w}$ and b, points that satisfy $\mathbf{w}^\intercal\mathbf{x} + b > 0$ are classified as positive, and points that satisfy $\mathbf{w}^\intercal\mathbf{x} + b < 0$ are classified as negative. The set $\mathbf{w}^\intercal\mathbf{x} + b = 0$ is the ***decision boundary***, which is a ***hyperplane***. The set of points on one side of the hyperplane is a ***half-space***. If it is possible to choose a linear decision boundary that correctly classifies all of the training cases, the training set is said to be ***linearly separable***.

Now assume there is no bias parameter. Each point in the ***weight space*** is a possible weight vector. Consider a positive training case $(\mathbf{x}, 1)$. The set of weight vectors which correctly classifies this training case is given by $\mathbf{w}^\intercal\mathbf{x} \geq 0$. For a lack of a better term, we will refer to the side which satisfies the constraint as the ***good region***. Similarly, for a negative training case $(\mathbf{x}, 0)$, the set of weight vectors which correctly classifies this training case is given by $\mathbf{w}^\intercal\mathbf{x} < 0$. The set of weight vectors which correctly classifies *all* of the training examples is the intersection of all the good regions of the individual examples. This set is called the ***feasible region***. If the feasible region is nonempty, the problem is said to be ***feasible***, otherwise it is ***infeasible***.

5.2 The Perceptron Learning Rule

To determine whether our binary linear classifier predicted correctly and update the weights correspondingly, we use the perceptron algorithm. This is different from gradient descent, as we do not use a loss function.

> **Perceptron algorithm**:
> For each training case $(\mathbf{x}^{(i)}, t^{(i)})$,
> $\quad z^{(i)} \leftarrow \mathbf{w}^\intercal\mathbf{x}^{(i)}$
> $\quad$ If $z^{(i)}t^{(i)} \leq 0$,
> $\qquad \mathbf{w} \leftarrow \mathbf{w} + t^{(i)}\mathbf{x}^{(i)}$

Let us look at how we came up with the perceptron algorithm. Consider the classification model as follows (We eliminated the bias b by adding a dummy dimension to $\mathbf{x}$):

$$z = \mathbf{w}^\intercal\mathbf{x}$$

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$$

In the perceptron algorithm, for each training case, we would like to determine if the prediction is correct. If the prediction is correct, we do nothing. If it is wrong, we adjust the weights in a direction that makes it more correct. In determining if the prediction is correct, since we don't want the training cases to lie on the decision boundary (as this means the classification may change if the input is perturbed even slightly), we use the stricter criterion $z^{(i)}t^{(i)} > 0$. This makes sense – if z and t have the same sign, then the prediction is correct. If they have different signs, then the prediction is wrong.

On adjusting the weight vector, consider the case of the training case being positive but we incorrectly classified it to be negative. We would like to increase the value of z, i.e. we would

like $z' = \mathbf{w}'^\intercal \mathbf{x} > \mathbf{w}^\intercal \mathbf{x} = z$ where $\mathbf{w}'$ and $\mathbf{w}$ are the new and old weight vectors respectively. The perceptron algorithm achieves this using the update $\mathbf{w}'^\intercal = \mathbf{w} + \alpha \mathbf{x}$ where $\alpha > 0$. Hence we have

$$\begin{aligned}\mathbf{w}'^\intercal \mathbf{x} &= (\mathbf{w} + \alpha \mathbf{x})^\intercal \mathbf{x} \\ &= \mathbf{w}^\intercal \mathbf{x} + \alpha \mathbf{x}^\intercal \mathbf{x} \\ &= \mathbf{w}^\intercal \mathbf{x} + \alpha ||\mathbf{x}||^2\end{aligned}$$

Since $||\mathbf{x}||^2$ is positive, we have $z' > z$. Conversely, if it is a negative training case which we incorrectly classified as positive, we use a negative α. It can be shown that the absolute value of α does not matter. Thus we can denote the update with

$$\mathbf{w} \leftarrow \mathbf{w} + t\mathbf{x}$$

which is known as the ***perceptron learning rule***.

5.3 The Limits of Linear Classifiers

A set S is ***convex*** if the line segment connecting any two points in S must lie within S. Any ***weighted average*** of points in S must also lie within S. A weighted average of points $\mathbf{x}^{(1)}, ..., \mathbf{x}^{(N)}$ is a point given by the linear combination

$$\mathbf{x}^{(avg)} = \lambda_1 \mathbf{x}^{(1)} + ... + \lambda_N \mathbf{x}^{(N)}$$

where $0 \leq \lambda_i \leq 1$ and $\lambda_1 + ... + \lambda_N = 1$. There are two sets that are always convex:

* The positive and negative regions in data space: Both regions are half-spaces. Thus if inputs $\mathbf{x}^{(1)}, ..., \mathbf{x}^{(N)}$ are all in the positive region, any weighted average must also be in the positive region.
* The feasible region in weight space: Each good region is convex because it is a half-space. Since the intersection of convex sets is convex, the feasible region is convex.

We can use convexity to show that functions are not linearly separable.

Example 5.3.1 Consider the *XOR* function. Since the positive region is convex, if we draw the line segment connecting the two positive examples (0, 1) and (1, 0), the entire line segment must be classified as positive. Similar for the line segment connecting the two negative examples (0, 0) and (1, 1). However, the line segments intersect at (0.5, 0.5), implying that (0.5, 0.5) is both positive and negative, which is impossible. Therefore, *XOR* is not linearly separable.

However, we could circumvent the problem using feature mapping. In place of $z = \mathbf{w}^\intercal \mathbf{x} + b$, we use $z = \mathbf{w}^\intercal \phi(\mathbf{x}) + b$, where $\phi(\mathbf{x}) = (\phi_1(\mathbf{x}), ..., \phi_D(\mathbf{x}))$ is a function mapping input vectors to feature vectors.

Example 5.3.2 Consider the XOR function, and consider the following feature representation for XOR:

$$\begin{aligned}\phi_1(\mathbf{x}) &= x_1 \\ \phi_2(\mathbf{x}) &= x_2 \\ \phi_3(\mathbf{x}) &= x_1 x_2\end{aligned}$$

In this representation, our training set becomes

$\phi_1(\mathbf{x})$	$\phi_2(\mathbf{x})$	$\phi_3(\mathbf{x})$	t
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Using the same techniques as Example 5.1, we find that $b = -0.5, w_1 = 1, w_2 = 1, w_3 = -2$ correctly classifies all the training examples.

5.4 Choosing a Cost Function for the Binary Classifier

To compute the gradient descent updates, we need to compute the partial derivatives $\partial\mathcal{L}/\partial w_j$. We will find ***surrogate loss functions***, which are loss functions we trust less than the other but which is easier to optimize.

5.4.1 0-1 loss

Choose the ***classification error*** loss or the ***0-1*** loss:

$$y = \begin{cases} 1 & \text{if } \mathbf{w}^\intercal\mathbf{x} > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\mathcal{L}_{0-1}(y, t) = \begin{cases} 0 & \text{if } y = t \\ 1 & \text{otherwise} \end{cases}$$

The cost $\mathcal{J}$ is the ***misclassification rate***:

$$\mathcal{J} = \frac{1}{N}\sum_{i=1}^{N} \delta[y^{(i)} \neq t^{(i)}]$$

As long as we are not on the classification boundary, making a small enough change to w_j will have *no effect* on $\mathcal{L}_{0-1}$, since the prediction won't change. This implies that $\partial\mathcal{L}_{0-1}/\partial w_j = 0$, as long as we are on the boundary. Further, the boundary is not differentiable. Thus we cannot optimize 0-1 loss with gradient descent.

5.4.2 Linear Regression

Choose the linear model and the squared error loss function:

$$y = \mathbf{w}^\intercal\mathbf{x} + b$$

$$\mathcal{L}_{\text{SE}}(y, t) = \frac{1}{2}(y - t)^2$$

We would pick some scheme to binarize the real-valued y, for example pick $y = 1/2$.

However, the problem is that if we have a positive target, i.e. $t = 1$. If we predict $y = 1$, we get a cost of 0, whereas if we make the wrong prediction $y = 0$, we get a cost of $1/2$. But if we predicted $y = 9$, then we pay a cost of $\frac{1}{2}(9 - 1)^2 = 32$, which is far higher than the cost for $y = 0$. There might be some sacrifices that the model makes to make up for this, but that would still end up incorrectly classifying other training examples.

5.4.3 Logistic Nonlinearity

Choose the squared error loss function, and apply a ***nonlinearity***, an ***activation function*** that squashes the predictions to be between 0 and 1:

$$z = \mathbf{w}^\intercal \mathbf{x} + b$$

$$y = \sigma(z)$$

$$\mathcal{L}_{\text{SE}}(y, t) = \frac{1}{2}(y - t)^2$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

is the ***logistic function***, a sigmoidal (S-shaped) function. This function has the feature that it increases monotonically, with asymptotes at 0 and 1.

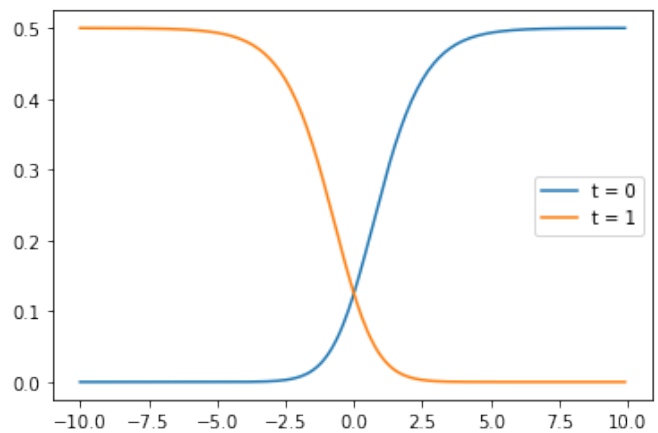

Figure 7: $\mathcal{L}_{\text{SE}}$ as a function of z

Let us derive the gradient descent updates. Note the convenient fact that

$$\frac{\partial y}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = y(1 - y)$$

So then we have

$$\frac{\mathrm{d}\mathcal{L}_{\text{SE}}}{\mathrm{d}z} = \frac{\mathrm{d}\mathcal{L}_{\text{SE}}}{\mathrm{d}y}\frac{\mathrm{d}y}{\mathrm{d}z} = (y - t)y(1 - y)$$

$$\frac{\partial \mathcal{L}_{\text{SE}}}{\partial w_j} = \frac{\mathrm{d}\mathcal{L}_{\text{SE}}}{\mathrm{d}z}\frac{\partial z}{\partial w_j} = \frac{\mathrm{d}\mathcal{L}_{\text{SE}}}{\mathrm{d}z} \cdot x_j$$

$$\frac{\partial \mathcal{L}_{\text{SE}}}{\partial b} = \frac{\mathrm{d}\mathcal{L}_{\text{SE}}}{\mathrm{d}z}\frac{\partial z}{\partial b} = \frac{\mathrm{d}\mathcal{L}_{\text{SE}}}{\mathrm{d}z}$$

However, as seen from Figure 7, the problem with this model is that the more confident the wrong prediction, the smaller the gradient, which implies that the most badly misclassified examples will have hardly any effect on the training, which is not good. We say that the algorithm does not have a strong ***gradient signal*** for these training examples. From the perspective of optimization, we would like bad predictions to be distinguished from very bad predictions.

5.4.4 Logistic Regression

Apply the logistic function and choose the ***cross-entropy*** loss function.

$$z = \mathbf{w}^\intercal \mathbf{x} + b$$

$$y = \sigma(z)$$

$$\mathcal{L}_{\text{CE}}(y, t) = \begin{cases} -\log y & \text{if } t = 1 \\ -\log(1-y) & \text{if } t = 0 \end{cases}$$

which can also be rewritten as

$$\mathcal{L}_{\text{CE}}(y, t) = -t \log y - (1-t)\log(1-y)$$

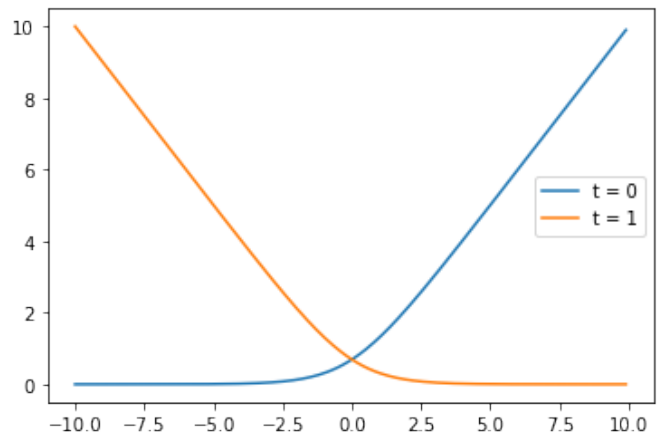

Figure 8: $\mathcal{L}_{\text{CE}}$ as a function of z

Let us now derive the gradient descent updates.

$$\frac{\mathrm{d}\mathcal{L}_{\text{CE}}}{\mathrm{d}y} = -\frac{t}{y} + \frac{1-t}{1-y}$$

$$\frac{\mathrm{d}\mathcal{L}_{\text{CE}}}{\mathrm{d}z} = \frac{\mathrm{d}\mathcal{L}_{\text{CE}}}{\mathrm{d}y}\frac{\mathrm{d}y}{\mathrm{d}z} = \frac{\mathrm{d}\mathcal{L}_{\text{CE}}}{\mathrm{d}y} \cdot y(1-y)$$

$$\frac{\mathrm{d}\mathcal{L}_{\text{CE}}}{\mathrm{d}w_j} = \frac{\mathrm{d}\mathcal{L}_{\text{CE}}}{\mathrm{d}z}\frac{\partial z}{\partial w_j} = \frac{\mathrm{d}\mathcal{L}_{\text{CE}}}{\mathrm{d}z} \cdot x_j = \left(-\frac{t}{y} + \frac{1-t}{1-y}\right) \cdot y(1-y) \cdot x_j = (y-t)x_j$$

Hence the gradient descent update for logistic regression is

$$\begin{aligned} w_j &\leftarrow w_j + \alpha \frac{\partial \mathcal{J}}{\partial w_j} \\ &= w_j - \frac{\alpha}{N}\sum_{i=1}^{N}(y^{(i)} - t^{(i)})x_j^{(i)} \end{aligned}$$

Notice that the gradient descent update rule is exactly same as the one for linear regression. Both logistic regression and linear regression are examples of ***generalized linear models***.

However, a problem with the gradient descent update and also the cross-entropy loss function itself is that if we confidently misclassify a positive example to be negative, so that $z \ll 0 \implies y \approx 0$, then y could be smaller than the smallest floating number, i.e. ***numerically zero***, then taking the log if 0 we get $-\infty$. Even if this does not happen, $\mathrm{d}\mathcal{L}_{\mathrm{CE}}/\,\mathrm{d}y$ will be extremely large in magnitude and again causing numerical difficulties. We come up with the ***logistic-cross-entropy***

$$\mathcal{L}_{\mathrm{LCE}}(z,t) = \mathcal{L}_{\mathrm{CE}}(\sigma(z),t) = t\log(1+e^{-z}) + (1-t)\log(1+e^{-z})$$

Most scientific computing environments provide a numerically stable **log-sum-exp** routine. In numpy, the following code would compute the logistic-cross-entropy:

```
E = t * np.logaddexp(0, -z) + (1 - t) * np.logaddexp(0, z)
```

Let us now compute the gradient descent update,

$$\begin{aligned}
\frac{\mathrm{d}\mathcal{L}_{\mathrm{LCE}}}{\mathrm{d}z} &= \frac{\mathrm{d}}{\mathrm{d}z}[t\log(1+e^{-z}) + (1-t)\log(1+e^{-z})] \\
&= -t \cdot \frac{e^{-z}}{1+e^{-z}} + (1-t)\frac{e^{z}}{1+e^{z}} \\
&= -t(1-y) + (1-t)y \\
&= y - t
\end{aligned}$$

which also has the intuitive interpretation: if $y > t$, then the prediction is too positive and we want to shift the prediction in the negative direction.

5.4.5 Support Vector Machine (SVM)

Use the ***hinge loss*** function.

$$y = \mathbf{w}^{\intercal}\mathbf{x} + b$$

$$\mathcal{L}_{\mathrm{H}} = \max(0, 1 - ty) \quad t \in \{-1, 1\}$$

This system is known as the ***Support Vector Machine***. One useful property is that it is an upper bound of 0-1 loss, which is a useful property for a surrogate loss function, since it means that if we make the hinge loss small, we have also made 0-1 loss small.

5.5 Multiclass Classifiction

Multiclass classification words by analogy with the binary case. We would represent targets using ***one-hot vector***, also called ***one-of-K encoding***:

$$\mathbf{t} = \underbrace{(0, ..., 0, 1, 0, ..., 0)}_{\text{entry } k \text{ is } 1}$$

The linear model is represented by

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

where $\mathbf{x}$ is a D-dimensional input vector, $\mathbf{z}$ is a K-dimensional output, $\mathbf{W}$ is a $K \times D$ ***weight matrix***, and $\mathbf{b}$ is a K-dimensional ***bias vector***.

The multivariate generalization of the logistic function $\sigma(\mathbf{z})$ is the ***softmax function***

$$y_i = \sigma(\mathbf{z})_i = \text{softmax}(z_1, ..., z_K)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{for } i = 1, ..., K$$

$\sigma(\mathbf{z})_i$ are nonnegative and sum to 1, so they can be interpreted as a probability distribution over the K classes. The inputs to the softmax are called the ***logits***. When one of the z_i's is much larger than the others, the output of the softmax will be approximately the argmax[7], in the one-hot encoding. Hence, a more accurate name might be "soft-argmax".

The multiple-output case of cross-entropy is given by

$$\mathcal{L}_{\text{CE}} = -\sum_{k=1}^{K} t_k \log y_k = -\mathbf{t}^\intercal(\log \mathbf{y})$$

where $\log \mathbf{y}$ represents the elementwise log. The loss function only makes sense for predictions which sum to 1; if we eliminate that constraint, we could trivially minimize the loss by making all the y_k's large.

Combining all the above, we have the ***multiclass logistic regression*** / ***softmax regression***

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{y} = \text{softmax}(\mathbf{z})$$

$$\mathcal{L}_{\text{CE}} = -\mathbf{t}^\intercal(\log \mathbf{y})$$

Similar to the case of single class classifiction, we can combine the softmax and cross-entropy functions into a single ***softmax-cross-entropy*** function $\mathcal{L}_{\text{SCE}}$. The derivatives are similar,

$$\frac{\partial \mathcal{L}_{\text{SCE}}}{\partial \mathbf{z}} = \mathbf{y} - \mathbf{t}$$

Hence we have our gradient updates:

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial \mathbf{w}_k} = \frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_k} \cdot \frac{\partial z_k}{\partial \mathbf{w}_k} = (y_k - t_k) \cdot \mathbf{x}$$

$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \alpha \frac{1}{N} \sum_{i=1}^{N} (y_k^{(i)} - t_k^{(i)}) \mathbf{x}^{(i)}$$

5.6 Convex Functions

Recall that a set S is convex if the line segment connecting any two points in S lies entirely within S. Mathematically, for $\mathbf{x}_0, \mathbf{x}_1 \in S$,

$$(1 - \lambda)\mathbf{x}_0 + \lambda \mathbf{x}_1 \in A \quad \text{for } 0 \leq \lambda \leq 1$$

[7] The argmax function operates on a vector and converts every value to zero except the maximum value, where it returns 1

A function f is a ***convex function*** if for any $\mathbf{x}_0, \mathbf{x}_1$ in the domain of f,

$$f((1-\lambda)\mathbf{x}_0 + \lambda\mathbf{x}_1) \leq (1-\lambda)f(\mathbf{x}_0) + \lambda f(\mathbf{x}_1)$$

Convexity is an important criterion when comparing different loss functions, because of two reasons:

1. All critical points are global minima, so if we can set the derivatives to zero, we have solved the problem.
2. Gradient descent will always converge to the global minimum

Although most of the optimization problems we consider in this course are non-convex, since training a deep neural network is a non-convex problem, even when the loss function is convex, convex loss functions still tend to be advantageous from the standpoint of optimization.

5.7 Tracking Accuracy for Binary Classification

The ***accuracy***, or the fraction correctly classified is defined as

$$\text{Accuracy } = \frac{TP+TN}{P+N} = \frac{TP+TN}{TP+TN+FP+FN}$$

where P, N, TP, TN, FP, FN represent number of positives, number of negatives, number of true positives, number of true negatives, number of false positve (type I error), and number of false negative (type II error).

$$\text{Sensitivity } = \frac{TP}{TP+FN} \quad \text{[True positive rate]}$$

$$\text{Specificity } = \frac{TN}{TN+FP} \quad \text{[True negative rate]}$$

Area under the ***Receiver Operating Characteristic (ROC) curve*** (AUC) is a useful metric to track if a binary classifier achieves a good tradeoff between sensitivity and specifity. For multiclass classification, we might also be interested in how frequently certain classes are confused. The ***confusion matrix*** is a $K \times K$ matrix, where rows are true labels, columns are predicted lables, and entries are frequencies.

5.8 Gradient Checking with Finite Differences

Gradient checking is important because algorithms often appear to work even if the math is wrong, but they can work better if the derivatives are correct. We could check if the gradients we derived are correct from the definition of the partial derivative The one-sided partial derivative is given by

$$\frac{\partial}{\partial x_i} f(x_1, ..., x_N) = \lim_{h \to 0} \frac{f(x_1, ..., x_i + h, ..., x_N) - f(x_1, ..., x_N)}{h}$$

The two-sided partial derivative is given by

$$\frac{\partial}{\partial x_i} f(x_1, ..., x_N) = \lim_{h \to 0} \frac{f(x_1, ..., x_i + h, ..., x_N) - f(x_1, ..., x_i - h, x_N)}{2h}$$

The two-sided definition is much more accurate than the one sided, and we can check the derivatives numerically by plugging in a small value of h, such as 10^{-10}, which is known as the method of ***finite differences***.

We then compute the ***relative error***:

$$\frac{|a-b|}{|a|+|b|}$$

where a is the finite differences estimate and b is the derivative computed by our implementation. The relative error should be very small.

5.9 Linear Classifiers v.s kNN

As a wrap-up for linear models, let us compare linear classifiers and kNN.

A ***hypothesis*** is a function $f : \mathcal{X} \to \mathcal{T}$ from the input to target space. The ***hypothesis space*** $\mathcal{H}$ is a set of hypotheses. For linear regression, $\mathcal{H}$ is the set of linear functions. A machine learning algorithm aims to find a good hypothesis $f \in \mathcal{H}$. An algorithm's ***inductive bias*** is the members of $\mathcal{H}$ and its preference for some hypotheses of $\mathcal{H}$ over others. The ***No Free Lunch*** Theorem states that if datasets or problems were not naturally biased, no machine learning algorithm would be better than another.

For a ***parametric*** algorithm, the hypothesis space $\mathcal{H}$ is defined using a finite set of parameters. Examples include linear regression, and logistic regression, as well as neural networks, k-means an Gaussian mixture models. For a ***non-parametric*** algorithm, the hypothesis space $\mathcal{H}$ is defined in terms of the data. Examples include k-nearest neighbours, decisions, Gaussian processes, and kernel estimation.

6 Neural Networks

To start off our discussion about neural networks, let us first look at a neuron in the brain. Neurons receive signals and accumulate voltage. After some threshold, they will fire spiking responses. For neural nets, we a much simpler model neuron, or ***unit***:

$$a = \phi\left(\sum_j w_j x_j + b\right)$$

where ϕ is the nonlinear activation function, w_j are the weights, b is the bias, a is the unit's activation.

We will represent complex nonlinear functions by connecting together lots of simple processing units into a ***neural network***, each of which computes a linear function, possibly followed by a nonlinearity. In aggregate, these units can compute some surprisingly complex functions. By historical accident, these networks are called ***multilayer perceptron***.

6.1 Multilayer Perceptrons

Looking at our model mentioned above, we have actually seen a bunch of examples of such units:

* Linear regression uses $\phi(z) = z$.
* In binaray linear classifiers, ϕ is a hard threshold at zero.
* In logistic regression, ϕ is the logistic function $\sigma(z) = 1/(1 + e^{-z})$.

A ***neural network*** is a combination of these units. We will focus on ***feed-forward neural networks***, where the units are arranged in a directed acyclic graph (DAG), so that all the computation can be done sequentially. This is in contrast with ***recurrent neural network***, where the graphs contain cycles and the processing can feed into itself.

The ***multilayer perceptron (MLP)*** is a kind of feed-forward network. The units are arranged into a set of ***layers***, and each layer contains some number of identical units. The network is ***fully connected*** – every unit in one layer is connected to every unit in the next layer. The ***input layer*** is the first layer, and its units take the values of the input features. The ***output layer*** is the last layer, and has one unit for each value the network outputs (i.e. a single unit in the case of regression or binary classification, or K units in the case of K-class classification). All the layers in between these are known as ***hidden layers***, because we don't know ahead of time what these units should compare, and this needs to be discovered during learning. The number of layers is known as the ***depth***, and the number of units in a layer is known as the ***width***. "Deep learning" refers to training neural nets with many layers.

Example 6.1.2 Let us consider the *XOR* function again. We can verbally describe *XOR* as "one of the inputs is 1, but not both of them." So we will have hidden unit h_1 detect if at least one of the inputs is 1, and h_2 detect if they are both 1. Then the output unit will actovate only if $h_1 = 1$ and $h_2 = 0$.

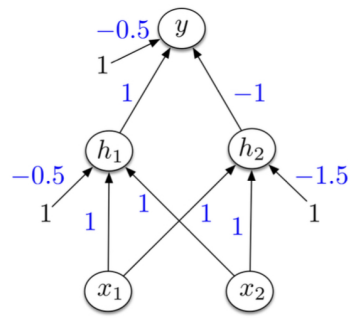

Figure 9: An MLP that computes the *XOR* function. All activation functions are binary thresholds at 0

The network's computations can be written out as:

$$h_i^{(1)} = \phi^{(1)}\left(\sum_j w_{ij}^{(1)} x_j + b_i^{(1)}\right)$$

$$h_i^{(2)} = \phi^{(2)}\left(\sum_j w_{ij}^{(2)} h_j^{(1)} + b_i^{(2)}\right)$$

$$y_i = \phi^{(3)}\left(\sum_j w_{ij}^{(3)} h_j^{(2)} + b_i^{(3)}\right)$$

noticing perhaps from Figure 9, that each unit has its own bias, and there is a weight for every pair of units in two consecutive layers. The i-th unit in the ℓ-th hidden layer is denoted $h_i^{(\ell)}$.

In vector form, we could represent the activations of all units in the ℓ-th layer with an ***activation vector*** $\mathbf{h}^{(l)}$. Each layer's weight is represented with a **weight matrix** $\mathbf{W}^{(\ell)}$. For hidden layer ℓ that connects $N_{\ell-1}$ hidden units to N_ℓ output units, the weight matrix is $N_\ell \times N_{\ell-1}$. Each layer also has a ***bias vector*** $\mathbf{b}^{(\ell)}$. Thus we can also write in vector form:

$$\mathbf{h}^{(1)} = \phi^{(1)}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right)$$

$$\mathbf{h}^{(2)} = \phi^{(2)}\left(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right)$$

$$\mathbf{y} = \phi^{(3)}\left(\mathbf{W}^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}\right)$$

where the activation function is appied independently to all the entries.

We can also store all of each layer's hidden units for all the training examples as a matrix $\mathbf{H}^{(\ell)}$, each row containing the hidden units for one example. The computations are written as follows:

$$\mathbf{H}^{(1)} = \phi^{(1)}\left(\mathbf{X}\mathbf{W}^{(1)\intercal} + \mathbf{1}\mathbf{b}^{(1)\intercal}\right)$$

$$\mathbf{H}^{(2)} = \phi^{(2)}\left(\mathbf{H}^{(1)}\mathbf{W}^{(2)\intercal} + \mathbf{1}\mathbf{b}^{(2)\intercal}\right)$$

$$\mathbf{Y} = \phi^{(3)}\left(\mathbf{H}^{(2)}\mathbf{W}^{(3)\intercal} + \mathbf{1}\mathbf{b}^{(3)\intercal}\right)$$

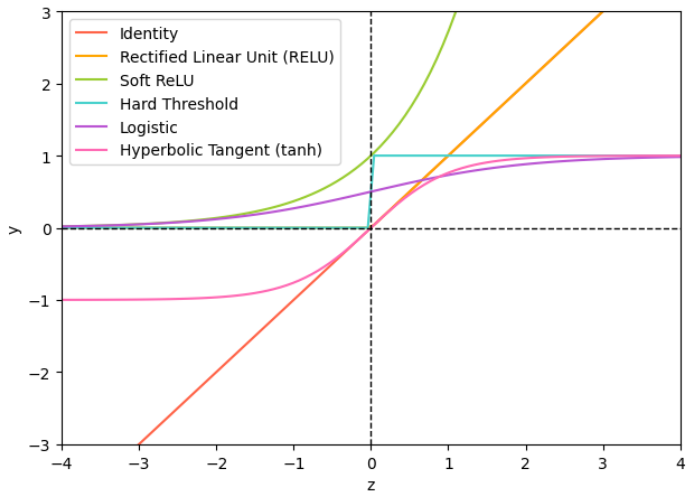

Figure 10: Some activation functions. Identity: $y = x$; ReLU: $y = \max(0, z)$; Soft ReLU: $y = \log 1 + e^z$; Hard Threshold; Logistic: $y = 1/(1 + e^z)$; Hyperbolic Tangent (tanh): $y = (e^z - e^{-z})/(e^z + e^{-z})$.

which can be translated into numpy code easily.

Some of the activation functions are shown in Figure 10.

Each layer computes a function, so the network computes a composition of functions:

$$\mathbf{y} = f^{(L)} \circ \cdots \circ f^{(1)}(\mathbf{x})$$

where in the last layer, if the task is regression, we choose

$$\mathbf{y} = f^{(L)}(\mathbf{h}^{(L-1)}) = \mathbf{w}^{(L)\intercal}\mathbf{h}^{(L-1)} + b^{(L)}$$

and if the task is binary classification, we choose

$$\mathbf{y} = f^{(L)}(\mathbf{h}^{(L-1)}) = \sigma(\mathbf{w}^{(L)\intercal}\mathbf{h}^{(L-1)} + b^{(L)})$$

6.2 Feature Learning

Static feature mappings are limited because it can be hard to design all the relevant features, and because the mappings might be impractically large. Neural nets can be though of as a way of learning nonlinear feature mappings. For example, a certain hidden layer can be thought of as a feature map $\boldsymbol{\psi}(\boldsymbol{x})$, and the output layer weights can be thought of as a linear model using those features. The whole network can be trained end-to-end with backpropagation (which will be discussed in a later section), and the goal is to learn a feature representation where the data becomes linearly separable.

Example 6.2.1 ...

6.3 Expressive Power

Linear models are limited in their expressive powers because it is difficult for them to expressive functions that are not linearly separable. For MLPs, the expressive power depends on the activation power.

For linear networks where we use the linear activation function $\phi(x) = x$, deep linear networks are no more powerful than shallow ones, since the networks function can just be expanded out as $\mathbf{y} = \mathbf{W}^{(L)}\mathbf{W}^{(L-1)} \cdots \mathbf{W}^{(1)}\mathbf{x}$, which could all be combined into one single matrix.

MLPs with nonlinear activation functions are ***universal***, because they can represent arbitrary functions.

Example 6.3.1 Consider binary inputs. The function can be given to us as a table which lists the output corresponding to every possible input vector. If there are D inputs, this table will have 2^D rows. Suppose the inputs are ± 1, and the hidden units use a hard threshold at 0. We use 2^D hidden units, each of which corresponds to one input configuration, and set the weight of an input to a hidden unit to be equal to the value of the input in that input configuration. In that way, only one hidden layer is required, though it is extremely wide.

Although the network is universal, not all functions can be represented compactly. We desire compact representations for two reasons:

* We want to be able to compute predictions in a reasonable amount of time.
* We want to be able to train a network to generalize from a limited number of training examples; universality implies that a large enough network can memorize the training set.

Same as before, a hard threshold is not convenient for learning the weights because the loss derivative is 0 everywhere. Using a soft threshold does not cause us in terms of a network's expressive power because we can approximate a hard threshold using a soft threshold just by scaling up the weights.

Since overfitting is a potential serious concern, we would need regularization. However, the topic of overfitting for neural networks is not well-understood, even by researchers. A common approach is early stopping, or stopping training early, because over-fitting typically increases as training progresses. Also, we don't add an explicit $\boldsymbol{\mathcal{R}(\theta)}$.

6.4 Back Propoagation

To train a multilayer neural network, we will use ***backpropagation***, which is a way of computing the partial derivatives of a loss function with respect to the parameters of a network, where the derivatives will then be used in gradient descent.

In calculating the derivatives, we will use the notation

$$\overline{v} = \frac{\partial \mathcal{L}}{\partial v}$$

and the multivariable chain rule will be written as

$$\overline{t} = \overline{x}\frac{\mathrm{d}x}{\mathrm{d}t} + \overline{y}\frac{\mathrm{d}y}{\mathrm{d}t}$$

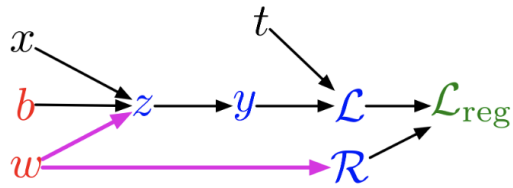

Figure 11: Computation graph for the regularized linear regression example

where we use $\mathrm{d}x/\,\mathrm{d}t$ to mean that we should actually evaluate the derivative algebraically, and $\overline{x}$ and $\overline{y}$ are values previously computed by the algorithm.

Now we will formally introduce the main algorithm for this set of notes, which is known as ***backpropagation***, or ***reverse mode automatic differentiation (autodiff)***.

Consider the example:

$$z = wx + b \qquad y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y-t)^2 \qquad \mathcal{R} = \frac{1}{2}w^2$$
$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda\mathcal{R}$$

and consider the ***computation graph*** in Figure 11.

The goal of backprop is to compute the derivatives $\overline{w}$ and $\overline{b}$. Notice from Figure 11 that when we compute the derivative of a variable, we need to first compute the derivatives for its children in the computation graph. The algorithm is as follows:

Let $v_1, ..., v_N$ denote all vertices in the computation graph, in a topologial ordering[8]. We wish to compute all of the derivatives $\overline{v_i}$, although we may only be interested in a subset of these values. We first compute all of the values in a ***forward pass***, then compute the derivatives in a ***backward pass***. As a special case, v_N denotes the result of the computation ($v_N = \mathcal{J}$ in our example) and is the variable we would like to compute the derivative of. By convention, we set $\overline{v_N} = 1$.

Algorithm (Back Propagation):
For i $= 1, ..., N$
    Compute v_i as a function of $\mathrm{Pa}(v_i)$
$v_N = 1$
For $i = N-1, ..., 1$
    $\overline{v_i} = \sum_{i \in \mathrm{Ch}(v_i)} \overline{v_j} \frac{\partial v_j}{\partial v_i}$

[8] Any ordering where the parent comes before children.

A Probability

Consider the example of flipping a coin twice,

Sample space Ω is the set of all possible outcomes of an experiment.(e.g. $\Omega = \{HH, HT, TH, TT\}$)

Observations $\omega \in \Omega$ are points in the space, also called sample outcomes, realizations, or elements. (e.g. $\omega = HT \in \Omega$)

Events $E \subset \Omega$ are subsets of the sample space. (e.g. the event where both flips have the same outcome $E = \{HH, TT\} \subset \Omega$)

Axioms of probability The probability of an event $E, P(E)$, satisfies three axioms:

1. $P(E) \geq 0$ for every E
2. $P(\Omega) = 1$
3. If $E_1, E_2, ...$ are disjoint then

$$P(\bigcup_{i=1}^{\infty} E_i) = \sum_{i=1}^{\infty} P(E_i)$$

Joint and conditional probability

$$P(A, B) = P(A|B)p(B) = P(B|A)P(A)$$

Independence Events A and B are independent if and only if $P(A, B) = P(A)P(B)$

Law of Total Probability

$$P(A) = \sum_{B} P(A, B) = \sum_{B} P(A|B)P(B)$$

Bayes' Rule

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \qquad \text{Posterior} = \frac{\text{Likelihood} * \text{Prior}}{\text{Evidence}}$$

Random variable A random variable is a mapping which assigns a real number $X(\omega)$ to each observed outcome $\omega \in \Omega$. A random variable is said to be independently and identically distributed (i.i.d.) if they are sampled from the same probability distribution and are mutually independent

Expectation value

$$\mathbb{E}[x] = \sum_{i=1}^{\infty} x_i p(x_i) \qquad \text{(univariate discreate R.V.)}$$

$$\mathbb{E}[x] = \int_{-\infty}^{\infty} x p(x) dx \qquad \text{(univariate continuous R.V.)}$$

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y] \qquad \mathbb{E}[cX] = x\mathbb{E}[X]$$

$$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y] \text{ only if } X \text{ and } Y \text{ are independent}$$

Variance

$$\mathrm{Var}[x] = \int_{-\infty}^{\infty} (x-\mu)^2 p(x)dx = \mathbb{E}[(x-\mu)^2] = \mathbb{E}[x^2] - (\mathbb{E}[x])^2$$

$$\mathrm{Var}[cX] = c^2\mathrm{Var}[X]$$

$$\mathrm{Var}[X+Y] = \mathrm{Var}[X] + \mathrm{Var}[Y] \text{ only if } X \text{ and } Y \text{ are independent}$$

Discrete and Continuous Random Variable

Discrete Random Variables

* Takes countably many values
* Distribution defined by probablity mass function(PMF)
* Marginalization: $p(x) = \sum_y p(x,y)$

Continuous Random Variables

* Takes uncountably many values
* Distribution defined by probablity density function(PDF)
* Marginalization: $p(x) = \int_y p(x,y)dy$

B Linear Algebra

Norms In general, the l_p norm is defined to be

$$||x||_p = \left(\sum_i |x_i|^p\right)^{1/p}$$

The l_2-norm is called the Euclidean norm: $\sqrt{\sum_i x_i^2}$.

The l_1-norm is called the Manhattan norm: $\sum_i |x_i|$.

The l_{inf}-norm is called the max norm: $\max_i |x_i|$.

C Regularized Linear Regression

We would like to determine the gradient descent update rules for the regularized cost function

$$\mathcal{J}_{\text{reg}}^{\alpha\beta}(\mathbf{w}) = \frac{1}{2N}\sum_{i=1}^{N}\left(y^{(i)} - t^{(i)}\right)^2 + \sum_{j=1}^{D}\alpha_j|w_j| + \frac{1}{2}\sum_{j=1}^{D}\beta_j w_j^2$$

Let

$$\mathcal{J} = \frac{1}{2N}\sum_{i=1}^{N}\left(y^{(i)} - t^{(i)}\right)^2 \qquad \mathcal{R}_1 = \sum_{j=1}^{D}\alpha_j|w_j| \qquad \mathcal{R}_2 = \frac{1}{2}\sum_{j=1}^{D}\beta_j w_j^2$$

The update rule for w_j is given by

$$\begin{aligned} w_j &\leftarrow w_j - \alpha \frac{\partial}{\partial w_j}(\mathcal{J} + \mathcal{R}_1 + \mathcal{R}_2) \\ &= w_j - \alpha(\frac{\partial \mathcal{J}}{\partial w_j} + \frac{\partial \mathcal{R}_1}{\partial w_j} + \frac{\partial \mathcal{R}_2}{\partial w_j}) \end{aligned}$$

Similarly, the update rule for b is given by

$$b \leftarrow b - \alpha(\frac{\partial \mathcal{J}}{\partial b} + \frac{\partial \mathcal{R}_1}{\partial b} + \frac{\partial \mathcal{R}_2}{\partial b})$$

Since

$$\mathcal{J} = \frac{1}{2N} \sum_{i=1}^{N} \left(\sum_{j=1}^{D} w_j x_j^{(1)} + b - t^{(i)} \right)^2,$$

we have

$$\begin{aligned} \frac{\partial \mathcal{J}}{\partial w_j} &= \frac{1}{2N} \sum_{i=1}^{N} 2x_j^{(i)} \left(\sum_{j'=1}^{D} w_{j'} x_{j'}^{(i)} + b - t^{(i)} \right) \quad \text{by chain rule} \\ &= \frac{1}{N} \sum_{i=1}^{N} x_j^{(i)} (y^{(i)} - t^{(i)}) \end{aligned}$$

$$\begin{aligned} \frac{\partial \mathcal{J}}{\partial b} &= \frac{1}{2N} \sum_{i=1}^{N} 2 \left(\sum_{j'=1}^{D} w_{j'} x_{j'}^{(i)} + b - t^{(i)} \right) \\ &= \frac{1}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)}) \end{aligned}$$

Consider $\mathcal{R}_2$.

$$\frac{\partial \mathcal{R}_2}{\partial b} = \frac{\partial}{\partial b} \frac{1}{2} (\beta_1 w_1^2 + ... + \beta_j w_j^2 + \beta_D w_D^2) = 0$$

$$\frac{\partial \mathcal{R}_2}{\partial w_j} = \frac{\partial}{\partial w_j} \frac{1}{2} (\beta_1 w_1^2 + ... + \beta_j w_j^2 + \beta_D w_D^2) = \beta_j w_j \quad \text{for } w_j < 0 \text{ and } w_j > 0$$

For $w_j = 0$, $\frac{\partial \mathcal{R}_2}{\partial w_j} = 0$.

Consider $\mathcal{R}_1$.

$$\frac{\partial \mathcal{R}_1}{\partial b} = \frac{\partial}{\partial w_j} (\alpha_1 |w_1| + ... + \alpha_j w_j + \alpha_D |w_D|) = 0$$

When $w_j > 0$, $|w_j| = w_j$. So

$$\frac{\partial \mathcal{R}_1}{\partial w_j} = \frac{\partial}{\partial w_j} (\alpha_1 |w_1| + ... + \alpha_j w_j + \alpha_D |w_D|) = \alpha_j$$

When $w_j = 0$, $|w_j| = 0$.

$$\frac{\partial \mathcal{R}_1}{\partial w_j} = \frac{\partial}{\partial w_j}(\alpha_1|w_1| + ... + \alpha_{j-1}w_{j-1} + \alpha_{j+1}w_{j+1} + \alpha_D|w_D|) = 0$$

When $w_j < 0$, $w_j = -z_j$ for some $z_j \in \mathbb{R}_{\geq 0}$. Then $|w_j| = z_j$.

$$\begin{aligned}\frac{\partial \mathcal{R}_1}{\partial w_j} &= \frac{\partial}{\partial w_j}(\alpha_1|w_1| + ... + \alpha_j z_j + \alpha_D|w_D|) \\ &= \frac{\partial}{\partial w_j}(\alpha_1|w_1| + ... - \alpha_j w_j + \alpha_D|w_D|) = -\alpha_j\end{aligned}$$

Therefore,

If $w_j > 0$:

$$w_j \leftarrow w_j - \alpha\left(\frac{1}{N}\sum_{i=1}^{N} x_j^{(i)}(y^{(i)} - t^{(i)}) + \alpha_j + \beta_j w_j\right)$$

$$b \leftarrow b - \alpha\left(\frac{1}{N}\sum_{i=1}^{N}(y^{(i)} - t^{(i)})\right)$$

If $w_j = 0$:

$$w_j \leftarrow w_j - \alpha\left(\frac{1}{N}\sum_{i=1}^{N} x_j^{(i)}(y^{(i)} - t^{(i)})\right)$$

$$b \leftarrow b - \alpha\left(\frac{1}{N}\sum_{i=1}^{N}(y^{(i)} - t^{(i)})\right)$$

If $w_j < 0$:

$$w_j \leftarrow w_j - \alpha\left(\frac{1}{N}\sum_{i=1}^{N} x_j^{(i)}(y^{(i)} - t^{(i)}) - \alpha_j + \beta_j w_j\right)$$

$$b \leftarrow b - \alpha\left(\frac{1}{N}\sum_{i=1}^{N}(y^{(i)} - t^{(i)})\right)$$

Suppose we only apply the ℓ_2 penalty and for simplicity, we drop the bias term from the model. We can solve this regularized regression problem directly by setting the partial derivatives to 0. We will obtain the form

$$\frac{\partial \mathcal{J}_{\text{reg}}^{\beta}}{\partial w_j} = \sum_{j'=1}^{D} A_{jj'}w_{j'} - c_j = 0$$

From the above, we have

$$\begin{aligned}
\frac{\partial \mathcal{J}_{\text{reg}}^{\beta}}{\partial w_j} &= \frac{1}{N}\sum_{i=1}^{N} x_j^{(i)} \left(\sum_{j'=1}^{D} w_{j'} x_{j'}^{(i)} - t^{(i)} \right) + \beta_j w_j \\
&= \frac{1}{N}\sum_{j'=1}^{D} \left(\sum_{i=1}^{N} x_j^{(i)} x_{j'}^{(i)} \right) w_{j'} - \frac{1}{N} x_j^{(i)} t^{(i)} + \beta_j w_j \\
&= \sum_{j'=1}^{D} \frac{1}{N} \left(\sum_{i=1}^{N} x_j^{(i)} x_{j'}^{(i)} \right) w_{j'} + \beta_j w_j - \frac{1}{N} x_j^{(i)} t^{(i)}
\end{aligned}$$

Comparing terms,

$$A_{jj'} = \begin{cases} \frac{1}{N}\sum_{i=1}^{N} x_j^{(i)} x_{j'}^{(i)} & \text{if} j \neq j' \\ \frac{1}{N}\sum_{i=1}^{N} x_j^{(i)} x_{j'}^{(i)} + \beta_j & \text{otherwise} \end{cases}$$

$$c_j = \frac{1}{N}\sum_{i=1}^{N} x_j^{(i)} t^{(i)}$$

$$\mathbf{A} = \begin{bmatrix} \frac{1}{N}\sum_{i=1}^{N} x_1^{(i)} x_1^{(i)} + \beta_1 & \cdots & \frac{1}{N}\sum_{i=1}^{N} x_1^{(i)} x_{j'}^{(i)} & \cdots & \frac{1}{N}\sum_{i=1}^{N} x_1^{(i)} x_D^{(i)} \\ \frac{1}{N}\sum_{i=1}^{N} x_2^{(i)} x_1^{(i)} & \ddots & \vdots & & \frac{1}{N}\sum_{i=1}^{N} x_2^{(i)} x_D^{(i)} \\ \vdots & & \frac{1}{N}\sum_{i=1}^{N} x_j^{(i)} x_j^{(i)} + \beta_j & & \vdots \\ \vdots & & \vdots & \ddots & \vdots \\ \frac{1}{N}\sum_{i=1}^{N} x_D^{(i)} x_1^{(i)} & \cdots & \frac{1}{N}\sum_{i=1}^{N} x_D^{(i)} x_{j'}^{(i)} & \cdots & \frac{1}{N}\sum_{i=1}^{N} x_D^{(i)} x_D^{(i)} + \beta_D \end{bmatrix}$$

$$\mathbf{c} = \begin{pmatrix} \frac{1}{N}\sum_{i=1}^{N} x_1^{(i)} t^{(i)} \\ \vdots \\ \frac{1}{N}\sum_{i=1}^{N} x_D^{(i)} t^{(i)} \end{pmatrix}$$

Since $\sum_{j'=1}^{D} A_{jj'} w_{j'}$ is multiplying the j'-th element of $\mathbf{c}$ to the j'-column elements of $\mathbf{A}$, it is the j'-th element of $\mathbf{Aw}$. We have

$$\begin{pmatrix} \sum_{j'=1}^{D} A_{1j'} w_{j'} - c_1 \\ \vdots \\ \sum_{j'=1}^{D} A_{jj'} w_{j'} - c_j \\ \vdots \\ \sum_{j'=1}^{D} A_{Dj'} w_{j'} - c_D \end{pmatrix} = \mathbf{Aw} - \mathbf{c} = \mathbf{0} \Rightarrow \mathbf{Aw} = \mathbf{c}$$

Since $\mathbf{A}$ is invertible, $\mathbf{w} = \mathbf{A}^{-1}\mathbf{c}$